ExamForge AI

Infrastructure Readiness Report

Generated: 2026-07-20 22:11 UTC

Classification: CONFIDENTIAL

Prepared by: Z.ai DevSecOps Team

Overall Score: 72/100

Adequate

1. Executive Summary

This Infrastructure Readiness Report evaluates ExamForge AI's operational maturity across eight critical dimensions: DevSecOps, Infrastructure Security, Monitoring, Logging, Alerting, Backup and Recovery, Deployment, and Operational Readiness. The assessment was conducted following the completion of a 12-phase infrastructure hardening initiative that transformed the platform from a development-stage application with minimal operational controls into a production-grade SaaS platform with enterprise-grade infrastructure.

The platform has undergone significant infrastructure improvements including the implementation of comprehensive CI/CD pipelines with automated security scanning, structured logging with sensitive data redaction, multi-tier backup and disaster recovery with defined RPO/RTO targets, infrastructure as code via Terraform, and complete operational documentation covering incident response, on-call procedures, and environment configuration. The overall score of 72/100 reflects a platform that is ready for limited production deployment (up to 100 schools) with specific areas requiring continued investment for enterprise scale (1,000+ schools).

2. Scoring Breakdown

Each dimension is scored on a 0-100 scale based on the implementation completeness, adherence to industry best practices, and verification through automated testing. A score of 70 or above is considered passing for production deployment.

Category	Score	Rating	Status
DevSecOps	75/100	Adequate	PASS
Infrastructure Security	72/100	Adequate	PASS
Monitoring	70/100	Adequate	PASS
Logging	78/100	Adequate	PASS
Alerting	68/100	Adequate	FAIL
Backup & Recovery	74/100	Adequate	PASS
Deployment	80/100	Good	PASS
Operational Readiness	65/100	Adequate	FAIL

Overall Weighted Score: 72/100

3. Detailed Findings

3.1 DevSecOps (75/100)

The DevSecOps implementation has been substantially improved with the introduction of comprehensive GitHub Actions CI/CD pipelines covering linting, testing, SAST, SCA, secret scanning, and build artifact verification. The CI pipeline enforces code quality gates, runs security-specific test suites, and verifies build integrity via SHA-256 checksums. The deployment pipeline requires manual approval for production, supports blue-green deployments, and includes automatic rollback on health check failure. However, some gaps remain: the SCA tooling relies on `dart pub outdated` rather than a dedicated dependency vulnerability database, and there is no container scanning since the application does not yet use Docker containers. The weekly scheduled security scan provides ongoing assurance.

3.2 Infrastructure Security (72/100)

Infrastructure security has been hardened with comprehensive HTTP security headers including CSP, HSTS, X-Frame-Options, and Permissions-Policy applied at the Caddy reverse proxy level. The Caddyfile enforces TLS 1.2+ with strong cipher suites, implements rate limiting per endpoint, and removes server identification headers. CORS is environment-specific with allow-lists per deployment target, replacing the previous wildcard configuration. The hardened .gitignore prevents secrets from being committed. Areas needing improvement include: no automated certificate monitoring, no HTTP to HTTPS redirect verification in CI, and the CSP policy allows `unsafe-inline` and `unsafe-eval` which is necessary for Flutter web but reduces XSS protection. A nonce-based CSP should be evaluated for future releases.

3.3 Monitoring (70/100)

A comprehensive monitoring schema has been implemented with dedicated tables for API latency metrics, AI service metrics, authentication metrics, payment metrics, server resource metrics, and storage metrics. Pre-built aggregation views provide 24-hour summaries for API latency (P50/P90/P95/P99), AI service performance (token usage, latency, quality scores), authentication events (failure rates, unique IPs), and payment processing (amounts, processing times). A health-check Edge Function continuously probes database, storage, auth, and payment services, recording results and evaluating alert rules. However, the monitoring currently relies on Supabase tables rather than a dedicated observability platform (e.g., Datadog, Grafana), which limits real-time dashboard capabilities and historical data retention for metrics older than the table retention period.

3.4 Logging (78/100)

The StructuredLogger replaces the original AppLogger with a production-ready logging system featuring JSON-formatted output, mandatory fields (timestamp, service, severity, correlation ID), four log channels (application, audit, security, payment), automatic sensitive data redaction, and context injection (user ID, school ID, request ID). The sensitive data redaction covers passwords, tokens, API keys, webhook hashes, card numbers, and CVVs. Bearer tokens and key-value patterns are automatically redacted in string content. The correlation ID system enables request tracing across service boundaries. The main gap is that the in-memory buffer (500 entries) requires a flush mechanism to external log aggregation (e.g., CloudWatch Logs via a scheduled upload). The current implementation logs critical errors via debugPrint in release mode, which is visible in crash

reports but not in a centralized log management system.

3.5 Alerting (68/100)

A complete alerting framework has been implemented with 18 pre-configured alert rules covering critical (database down, payment provider down, security breach), high (error rates, latency spikes, backup failures), and warning (CPU/memory spikes, slow queries, prompt injection attempts) categories. Escalation policies define response channels and timeouts per severity level, from on-call Slack notification to CTO escalation. The alert_state table tracks firing alerts with cooldown management to prevent alert fatigue. The alert_history table maintains a complete audit trail. However, the alerting currently relies on the health-check Edge Function for evaluation, which runs on-demand rather than on a guaranteed schedule. A scheduled cron trigger (e.g., via Supabase pg_cron or an external scheduler) is needed for reliable alert evaluation. Additionally, Slack webhook integration is configured in the notification_channels table but the actual webhook URLs need to be provisioned.

3.6 Backup and Recovery (74/100)

The backup and disaster recovery system has been significantly enhanced with the backup_dr.sh script supporting database, configuration, and storage backups with GPG encryption, SHA-256 integrity verification, and S3 upload with cross-region replication (af-south-1 primary, eu-west-1 DR). RPO is set at 1 hour and RTO at 4 hours, with the recovery test feature enabling regular verification. The Terraform infrastructure defines encrypted S3 buckets with versioning, lifecycle policies (30-day daily, 90-day Standard-IA, 365-day Glacier), and least-privilege IAM policies. The retention policy maintains 30 days of daily backups and 12 months of monthly backups. Remaining gaps include: no automated scheduling (cron jobs not yet configured), WAL archiving not enabled for true incremental backups, and recovery testing should be automated on a monthly schedule. The backup role (backup_reader) has been created with read-only access for pg_dump operations.

3.7 Deployment (80/100)

The deployment infrastructure scores highest due to the mature deploy.sh script that already supported multi-environment deployment, blue-green switching, automatic rollback, and health checking. This has been complemented with GitHub Actions workflows for CI (build, test, security scan) and CD (deployment with approval gates, artifact verification, post-deployment smoke tests). The deploy.yml workflow requires manual approval for production via GitHub Environments, verifies build artifact integrity via SHA-256 checksums, and performs post-deployment health checks on both the application and API endpoints. The weekly security scan workflow provides ongoing assurance. The existing deploy.sh retains its value for manual deployments and environments where GitHub Actions is not available. Improvements needed: no Docker containerization (deployment is via rsync), no canary deployment support, and the migration tracking table should use advisory locks to prevent concurrent migration execution.

3.8 Operational Readiness (65/100)

Operational readiness has been addressed through seven comprehensive documentation files covering deployment, incident response, backup and restore, monitoring, security operations, on-call procedures, and environment configuration. The operational test suite (operational_test.sh) provides automated verification of failure detection and recovery procedures. The operational

security implementation defines six database roles with scoped permissions (webhook_processor, refund_processor, monitoring_agent, backup_reader, analytics_reader), storage bucket access policies, administrative access audit logging, and a dual-approval system for sensitive operations. The remaining gaps are significant: no actual on-call rotation has been established, no incident has been simulated end-to-end, the operational tests verify infrastructure exists but do not simulate actual failures in a running environment, and the runbook procedures have not been validated through tabletop exercises. These gaps are expected for a platform transitioning from development to initial production deployment.

4. Remaining Issues and Remediation

The following table summarizes the remaining issues identified during this assessment, organized by severity. Each issue includes the recommended remediation and current status. Critical and High severity items should be addressed before the platform serves more than 100 schools.

Severity	Finding	Remediation	Status
Critical	No containerized deployment	Docker + container registry	Open
Critical	No automated backup scheduling	Configure cron/pg_cron	Open
High	No external log aggregation	Integrate CloudWatch/Datadog	Open
High	CSP allows unsafe-inline/eval	Implement nonce-based CSP	Open
High	No alert evaluation scheduler	Configure pg_cron trigger	Open
High	SCA lacks dedicated vuln DB	Integrate Snyk/Dependabot	Open
Medium	No canary deployment support	Add canary analysis to pipeline	Open
Medium	WAL archiving not enabled	Enable PostgreSQL WAL archiving	Open
Medium	No certificate monitoring	Add cert expiry alerting	Open
Medium	Operational tests not validated	Run tabletop exercises	Open
Low	No on-call rotation established	Set up PagerDuty/rotation	Open
Low	No incident simulation done	Schedule quarterly game days	Open

5. Launch Recommendation

Based on the infrastructure readiness assessment, the following launch recommendations are provided for each scale tier. These recommendations consider the current implementation state, remaining gaps, and the operational risk associated with each deployment scale.

Scale	Recommendation	Confidence	Key Blockers
10 Schools	APPROVED	High	Configure backup scheduling; Set up Slack webhooks
100 Schools	APPROVED with conditions	Medium	Containerize deployment; External log aggregation; Alert sched
1,000 Schools	NOT RECOMMENDED	Low	Canary deployments; Horizontal scaling; Dedicated observabilit

10 Schools: The platform is ready for a limited production launch serving up to 10 schools. The security infrastructure (constant-time comparison, AES-256-GCM encryption, server-side refund validation, AI prompt injection detection, CORS hardening) is in place. The CI/CD pipeline ensures code quality and prevents secret leaks. Monitoring and alerting frameworks are defined. The

primary gap is operational: backup scheduling and notification channel provisioning must be completed before launch. These can be addressed within one sprint.

100 Schools: Launch is approved conditional on completing three additional items: containerizing the deployment for consistent environments and easier scaling, integrating an external log aggregation service for centralized observability, and configuring a scheduled alert evaluation mechanism (pg_cron or external scheduler). These items represent a 2-4 week engineering investment. The structured logging and monitoring schema are ready to feed into these systems.

1,000 Schools: Launch is not recommended at this scale. The platform lacks horizontal scaling architecture, canary deployment capability, a dedicated observability platform with real-time dashboards, an established on-call rotation, and validated incident response procedures through tabletop exercises. Reaching 1,000-school readiness requires a dedicated SRE investment of 2-3 months, including load testing at scale, database connection pool optimization for high concurrency, CDN integration for static assets, and possibly a migration to a container orchestration platform (Kubernetes).

6. Implementation Summary

Phase	Deliverable	Status
1. CI/CD Hardening	GitHub Actions workflows (ci.yml, deploy.yml, security-scan.yml)	Complete
2. Environment & Secrets	ENVIRONMENT_REFERENCE.md, hardened .gitignore, rotation schedule	Complete
3. Infrastructure Hardening	Security headers, CSP, HSTS, Caddyfile, CORS per environment	Complete
4. Monitoring & Observability	7 metrics tables, 4 aggregation views, health-check Edge Function	Complete
5. Structured Logging	StructuredLogger with 4 channels, sensitive data redaction, correlation IDs	Complete
6. Alerting	18 alert rules, escalation policies, notification channels, alert history	Complete
7. Backup & DR	backup_dr.sh with RPO=1h, RTO=4h, encryption, cross-region replication	Complete
8. Infrastructure as Code	Terraform main.tf (S3, IAM, CloudWatch Logs, DR bucket)	Complete
9. Production Deployment	CI/CD with approval gates, blue-green, health checks, artifact verification	Complete
10. Operational Security	6 DB roles, storage policies, admin audit log, operation approval	Complete
11. Operational Documentation	7 runbooks/guides (23,000+ words)	Complete
12. Operational Testing	operational_test.sh with 8 failure scenarios	Complete